

Shell Implementation Documentation

This document provides a detailed explanation of a custom shell implementation written in C. The shell supports basic command execution, pipes, I/O redirection, command history, and built-in commands.

Table of Contents

1. [Overview](#)
2. [Header Files](#)
3. [Constants and Global Variables](#)
4. [Main Function](#)
5. [Main Loop](#)
6. [Signal Handling](#)
7. [Command Parsing](#)
8. [Command Execution](#)
9. [Pipeline Execution](#)
10. [Helper Functions](#)

Overview

This shell implementation provides a command-line interface similar to common Unix shells like Bash. It supports:

- Command execution with arguments
- Input/output redirection (`<` , `>` , `>>`)
- Command pipelines (`|`)
- Command operators (`;` , `&&` , `||`)
- Built-in commands (`cd` , `exit` , `history` , `clr` , `help`)
- Command history tracking
- Signal handling (SIGINT/Ctrl+C)

Header Files

```
#include <stdio.h>      // Standard I/O functions (printf, fgets)
#include <stdlib.h>     // Memory allocation, exit
#include <string.h>     // String manipulation functions
```

```
#include <unistd.h>      // POSIX operating system API (chdir, fork, execvp)
#include <fcntl.h>        // File control options
#include <sys/wait.h>     // Process wait functions
#include <signal.h>       // Signal handling
#include <errno.h>        // Error number definitions
#include <sys/stat.h>     // File status and attributes
```

Each header provides essential functions:

- `stdio.h` : Input/output operations like `printf` and `fgets`
- `stdlib.h` : Memory management and process control
- `string.h` : String manipulation functions like `strcmp` and `strdup`
- `unistd.h` : POSIX API functions for system calls
- `fcntl.h` : File manipulation and control
- `sys/wait.h` : Functions for waiting on processes
- `signal.h` : Signal handling capabilities
- `errno.h` : Error code definitions
- `sys/stat.h` : File status and attribute functions

Constants and Global Variables

```
#define MAX_LINE_LEN 1024 // Maximum length of input command line
#define MAX_ARGS 128      // Maximum number of arguments for a command
#define MAX_HISTORY 100   // Maximum number of commands to store in history

#define OP_SEQ 0           // Sequential command operator (;)
#define OP_AND 1          // Logical AND operator (&&)
#define OP_OR 2           // Logical OR operator (||)

static char *history[MAX_HISTORY]; // Array to store command history
static int history_count = 0;       // Counter for number of commands in history
```

These constants define the limits of the shell:

- `MAX_LINE_LEN` : Limits input command length to 1024 characters
- `MAX_ARGS` : Limits number of arguments to 128
- `MAX_HISTORY` : Limits history to 100 commands
- Operator constants define the types of command operators supported
- The `history` array stores previous commands
- `history_count` tracks how many commands are in the history

Main Function

```
int main(void)
{
    umask(0);                // Set file creation mask to 0 (full permissions)
    struct sigaction sa;     // Declare signal action structure
    sa.sa_handler = handle_sigint; // Set signal handler function
    sigemptyset(&sa.sa_mask); // Initialize signal mask to empty
    sa.sa_flags = SA_RESTART; // Set flag to restart interrupted system calls
    sigaction(SIGINT, &sa, NULL); // Register handler for SIGINT (Ctrl+C)
    main_loop();             // Start the shell's main loop
    for (int i = 0; i < history_count; i++) {
        free(history[i]);    // Free memory allocated for history items
    }
    return 0;                // Return success status
}
```

The main function:

1. Sets the file creation mask to 0 (allowing full permissions)
2. Sets up signal handling for SIGINT (Ctrl+C) using `sigaction`
3. Enters the main shell loop with `main_loop()`
4. Cleans up memory used by the command history before exiting
5. Returns a success status code (0)

Main Loop

```
void main_loop(void)
{
    while (1) {              // Infinite loop until exit
        char line[MAX_LINE_LEN]; // Buffer for input command
        printf("sh> ");        // Print shell prompt
        fflush(stdout);         // Ensure prompt is displayed immediately
        if (!fgets(line, sizeof(line), stdin)) { // Read input, check for EOF
            printf("\n");      // Print newline on EOF
            break;             // Exit loop if EOF
        }
        size_t ln = strlen(line); // Get length of input line
        if (ln > 0 && line[ln-1] == '\n') {
            line[ln-1] = '\0'; // Remove trailing newline
        }
        if (strcmp(line, "") == 0) { // Skip empty commands
            continue;
        }
    }
}
```

```

    }
    add_to_history(line);           // Add command to history
    parse_line(line);             // Parse and execute the command
}
}

```

The `main_loop` function:

1. Runs an infinite loop until exited with Ctrl+D or `exit` command
2. Displays the shell prompt (`sh>`)
3. Reads a command line from standard input
4. Handles EOF (Ctrl+D) by breaking out of the loop
5. Removes the trailing newline character
6. Skips empty commands
7. Adds valid commands to the history
8. Calls `parse_line()` to process and execute the command

Signal Handling

```

void handle_sigint(int signo)
{
    write(STDOUT_FILENO, "\nsh> ", 5); // Write new prompt after Ctrl+C
    fflush(stdout);                     // Flush output buffer
}

```

The `handle_sigint` function:

1. Handles SIGINT signal (Ctrl+C)
2. Writes a new shell prompt to the terminal
3. Flushes the output buffer to ensure prompt is displayed
4. This allows the shell to continue running when Ctrl+C is pressed

Command Parsing

```

void parse_line(char *line)
{
    char *cmd_list[128];           // Array to store individual commands
    int operators[128];            // Array to store operators between commands
    int count = 0;                 // Command counter
    memset(cmd_list, 0, sizeof(cmd_list)); // Initialize command list to NULL
}

```

```

memset(operators, -1, sizeof(operators)); // Initialize operators to -1
char *start = line;                       // Pointer to start of current command

// Parsing loop to separate commands by operators
while (*start) {
    char *sc = strchr(start, ';');        // Find semicolon operator
    char *amp = strstr(start, "&&");      // Find AND operator
    char *or_ = strstr(start, "||");      // Find OR operator
    char *delim = NULL;                   // Pointer to closest delimiter
    int op_type = -1;                     // Type of operator found

    // Find the closest operator
    if (sc && (!delim || sc < delim)) {
        delim = sc;
        op_type = OP_SEQ;                 // Sequential operator
    }
    if (amp && (!delim || amp < delim)) {
        delim = amp;
        op_type = OP_AND;                 // AND operator
    }
    if (or_ && (!delim || or_ < delim)) {
        delim = or_;
        op_type = OP_OR;                  // OR operator
    }

    // Handle case with no more operators
    if (!delim) {
        cmd_list[count++] = strdup(start); // Copy remaining command
        break;
    } else {
        // Extract command before operator
        int length = delim - start;
        char tmp[1024];
        strncpy(tmp, start, length);
        tmp[length] = '\0';
        cmd_list[count] = strdup(tmp);     // Store command
        operators[count] = op_type;        // Store operator type
        count++;

        // Move pointer past the operator
        if (op_type == OP_AND || op_type == OP_OR) {
            start = delim + 2;              // Move past && or ||
        } else {
            start = delim + 1;              // Move past ;
        }
        while (*start == ' ') start++;     // Skip spaces
    }
}

```

```

    if (count == 0) return; // No commands to execute

    // Execute the parsed commands
    execute_commands(cmd_list, count, operators);

    // Free allocated memory
    for (int i = 0; i < count; i++) {
        free(cmd_list[i]);
    }
}

```

The `parse_line` function:

1. Initializes arrays to store commands and operators
2. Iteratively finds command separators (; , && , ||)
3. Extracts each command and identifies the operator type
4. Allocates memory for each command with `strdup`
5. Tracks operator types between commands
6. Calls `execute_commands()` to run the parsed commands
7. Frees allocated memory for command strings

Command Execution

```

int execute_commands(char **cmd_list, int cmd_count, int *operators)
{
    int status = 0; // Return status of last command
    for (int i = 0; i < cmd_count; i++) {
        if (i > 0) {
            int prev_op = operators[i - 1]; // Get previous operator
            if (prev_op == OP_AND && status != 0) {
                continue; // Skip if AND and previous failed
            } else if (prev_op == OP_OR && status == 0) {
                continue; // Skip if OR and previous succeeded
            }
        }
        status = execute_single_command(cmd_list[i]); // Execute command
    }
    return status; // Return last command's status
}

```

The `execute_commands` function:

1. Processes a list of commands separated by operators

2. Implements logical operator behavior:

- ; (sequential): Always execute next command
- && (AND): Execute next command only if previous succeeded (status = 0)
- || (OR): Execute next command only if previous failed (status != 0)

3. Returns the status of the last executed command

```
int execute_single_command(char *cmd)
{
    char *commands[64];                // Array for commands in a pipeline
    int n = 0;                          // Counter for pipeline segments
    commands[n++] = strtok(cmd, "|");    // Get first command in pipeline
    while (1) {
        char *p = strtok(NULL, "|");    // Get next command in pipeline
        if (!p) break;                  // End of pipeline
        commands[n++] = p;
    }
    commands[n] = NULL;                 // Null-terminate array

    if (n > 1) return execute_pipeline(commands, n); // Handle pipeline

    // Process a single command (no pipeline)
    {
        char *single = commands[0];
        if (!single) return 0;
        char temp_buf[1024];
        strncpy(temp_buf, single, sizeof(temp_buf)); // Copy command for tokenization
        temp_buf[sizeof(temp_buf)-1] = '\0';
        char *args[MAX_ARGS];                // Array for command arguments
        int arg_count = 0;                    // Counter for arguments
        int in_redir = 0, out_redir = 0, append_redir = 0; // Redirection flags
        char *infile = NULL, *outfile = NULL; // Redirection filenames

        // Parse command arguments and redirections
        char *token = strtok(temp_buf, " \t");
        while (token) {
            if (strcmp(token, "<") == 0) { // Input redirection
                in_redir = 1;
                token = strtok(NULL, " \t");
                if (token) infile = token;
            } else if (strcmp(token, ">") == 0) { // Output redirection
                out_redir = 1;
                token = strtok(NULL, " \t");
                if (token) outfile = token;
            } else if (strcmp(token, ">>") == 0) { // Append redirection
                append_redir = 1;
                token = strtok(NULL, " \t");
                if (token) outfile = token;
            }
        }
    }
}
```

```

    } else {
        strip_surrounding_quotes(token);    // Remove quotes from argument
        args[arg_count++] = token;         // Store argument
    }
    token = strtok(NULL, " \t");           // Get next token
}
args[arg_count] = NULL;                   // Null-terminate args array

if (arg_count == 0) {
    return 0;                             // Empty command
}

// Handle built-in commands
if (strcmp(args[0], "cd") == 0 ||
    strcmp(args[0], "exit") == 0 ||
    strcmp(args[0], "history") == 0 ||
    strcmp(args[0], "clr") == 0 ||
    strcmp(args[0], "help") == 0) {

    if (strcmp(args[0], "cd") == 0) {
        // Change directory command
        if (!args[1]) {
            char *home = getenv("HOME");    // Go to home if no arg
            if (home) {
                if (chdir(home) != 0) {
                    perror("cd");
                    return 1;
                }
            }
        } else {
            if (chdir(args[1]) != 0) {        // Change to specified dir
                perror("cd");
                return 1;
            }
        }
        return 0;
    } else if (strcmp(args[0], "exit") == 0) {
        exit(0);                            // Exit the shell
    } else if (strcmp(args[0], "history") == 0) {
        // Show command history
        for (int i = 0; i < history_count; i++) {
            printf("%d %s\n", i+1, history[i]);
        }
        return 0;
    } else if (strcmp(args[0], "clr") == 0) {
        system("clear");                     // Clear screen
        return 0;
    } else if (strcmp(args[0], "help") == 0) {
        // Display help information
    }
}

```



```

        printf("Built-in commands:\n");
        printf("cd [dir]          Change directory (no quotes required)\n");
        printf("exit              Exit the shell\n");
        printf("history          Show command history\n");
        printf("clr              Clear the terminal screen\n");
        printf("help             Display this help message and list common system commands\n");
        printf("Common system commands available:\n");
        // [Additional help text omitted for brevity]
        return 0;
    }
}

// Execute external command
pid_t pid = fork(); // Create child process
if (pid < 0) {
    perror("fork");
    return 1;
}
if (pid == 0) { // Child process
    signal(SIGINT, SIG_DFL); // Reset signal handler

    // Handle input redirection
    if (in_redir && infile) {
        int fd_in = open(infile, O_RDONLY);
        if (fd_in < 0) {
            perror("open for input");
            exit(1);
        }
        dup2(fd_in, STDIN_FILENO); // Redirect stdin
        close(fd_in);
    }

    // Handle output redirection
    if (out_redir && outfile) {
        int fd_out = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0777);
        if (fd_out < 0) {
            perror("open for output");
            exit(1);
        }
        dup2(fd_out, STDOUT_FILENO); // Redirect stdout
        close(fd_out);
    } else if (append_redir && outfile) {
        int fd_out = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0777);
        if (fd_out < 0) {
            perror("open for append");
            exit(1);
        }
        dup2(fd_out, STDOUT_FILENO); // Redirect stdout (append)
        close(fd_out);
    }
}

```

```

    }

    execvp(args[0], args);           // Execute command
    perror("execvp");               // Only reached if execvp fails
    exit(1);
}

// Parent process
int status = 0;
waitpid(pid, &status, 0);           // Wait for child to finish
if (WIFEXITED(status)) {
    return WEXITSTATUS(status);     // Return child's exit status
}
return 1;                           // Child didn't exit normally
}
}

```

The `execute_single_command` function:

1. Tokenizes the command by pipe (|) characters to identify pipeline segments
2. Redirects to `execute_pipeline()` if multiple pipeline segments are found
3. For single commands:
 - Parses arguments and redirection operators (< , > , >>)
 - Handles built-in commands (cd , exit , history , clr , help)
 - Creates a child process with `fork()` for external commands
 - Sets up input/output redirection in the child process
 - Executes the command with `execvp()`
 - Waits for the child process to complete and returns its exit status

Pipeline Execution

```

int execute_pipeline(char **commands, int n)
{
    int pipefd[2 * (n - 1)];          // Array of pipe file descriptors
    for (int i = 0; i < n - 1; i++) {
        if (pipe(pipefd + 2 * i) < 0) { // Create pipes
            perror("pipe");
            return 1;
        }
    }

    int status = 0;
    for (int i = 0; i < n; i++) {      // Loop for each command in pipeline

```

```

pid_t pid = fork(); // Create child process
if (pid < 0) {
    perror("fork");
    return 1;
} else if (pid == 0) { // Child process
    signal(SIGINT, SIG_DFL); // Reset signal handler

    // Set up pipe connections
    if (i > 0) // Not first command: read from previous pipe
        dup2(pipefd[2 * (i - 1)], STDIN_FILENO);
    if (i < n - 1) // Not last command: write to next pipe
        dup2(pipefd[2 * i + 1], STDOUT_FILENO);

    // Close all pipe file descriptors
    for (int j = 0; j < 2 * (n - 1); j++)
        close(pipefd[j]);

    // Parse command arguments and handle redirections (similar to execute_single_command)
    char *segment = commands[i];
    char *args[MAX_ARGS];
    int arg_count = 0;
    int in_redir = 0, out_redir = 0, append_redir = 0;
    char *infile = NULL, *outfile = NULL;

    char *tok = strtok(segment, " \t");
    while (tok) {
        if (strcmp(tok, "<") == 0) { // Input redirection
            in_redir = 1;
            tok = strtok(NULL, " \t");
            if (tok) infile = tok;
        } else if (strcmp(tok, ">") == 0) { // Output redirection
            out_redir = 1;
            tok = strtok(NULL, " \t");
            if (tok) outfile = tok;
        } else if (strcmp(tok, ">>") == 0) { // Append redirection
            append_redir = 1;
            tok = strtok(NULL, " \t");
            if (tok) outfile = tok;
        } else {
            strip_surrounding_quotes(tok); // Remove quotes
            args[arg_count++] = tok; // Store argument
        }
        tok = strtok(NULL, " \t"); // Get next token
    }
    args[arg_count] = NULL; // Null-terminate args array

    if (arg_count == 0) {
        exit(0); // Empty command
    }
}

```

```

// Handle redirections for this pipeline segment
if (in_redir && infile) {
    int fd_in = open(infile, O_RDONLY);
    if (fd_in < 0) {
        perror("open for input");
        exit(1);
    }
    dup2(fd_in, STDIN_FILENO);          // Redirect stdin
    close(fd_in);
}

if (out_redir && outfile) {
    int fd_out = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0777);
    if (fd_out < 0) {
        perror("open for output");
        exit(1);
    }
    dup2(fd_out, STDOUT_FILENO);        // Redirect stdout
    close(fd_out);
} else if (append_redir && outfile) {
    int fd_out = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0777);
    if (fd_out < 0) {
        perror("open for append");
        exit(1);
    }
    dup2(fd_out, STDOUT_FILENO);        // Redirect stdout (append)
    close(fd_out);
}

execvp(args[0], args);                  // Execute command
perror("execvp");                       // Only reached if execvp fails
exit(1);
}
}

// Parent process
for (int i = 0; i < 2 * (n - 1); i++)    // Close all pipe fds in parent
    close(pipefd[i]);

for (int i = 0; i < n; i++) {            // Wait for all children
    int child_status = 0;
    wait(&child_status);
    if (WIFEXITED(child_status)) {
        status = WEXITSTATUS(child_status); // Get last exit status
    } else {
        status = 1;                        // Child didn't exit normally
    }
}
}

```

```

    return status;                                // Return last command's status
}

```

The `execute_pipeline` function:

1. Creates $n-1$ pipes for n commands in the pipeline
2. Creates a child process for each command
3. In each child process:
 - Connects the process to appropriate pipe ends (read from previous, write to next)
 - Parses command arguments and handles redirections
 - Executes the command with `execvp()`
4. In the parent process:
 - Closes all pipe file descriptors
 - Waits for all child processes to complete
 - Returns the exit status of the last command

Helper Functions

```

void strip_surrounding_quotes(char *str)
{
    size_t len = strlen(str);                // Get string length
    if (len < 2)                             // Too short for quotes
        return;
    char first = str[0];                     // First character
    char last = str[len - 1];                // Last character
    if ((first == '"' && last == '"') || (first == '\'' && last == '\'')) {
        memmove(str, str + 1, len - 1);      // Remove first quote
        str[len - 2] = '\0';                 // Remove last quote and terminate
    }
}

```

The `strip_surrounding_quotes` function:

1. Detects if a string is surrounded by matching quotes (single or double)
2. Removes the quotes while preserving the content inside
3. Uses `memmove` to efficiently shift the string contents

```

void add_to_history(const char *cmd)
{
    if (history_count < MAX_HISTORY) {        // History not full
        history[history_count++] = strdup(cmd); // Add command to history
    }
}

```

```

    } else {
        free(history[0]); // Free oldest command
        for (int i = 1; i < MAX_HISTORY; i++) {
            history[i - 1] = history[i]; // Shift history entries
        }
        history[MAX_HISTORY - 1] = strdup(cmd); // Add new command at end
    }
}

```

The `add_to_history` function:

1. Adds a command to the history list
2. If history is not full, appends the command
3. If history is full, removes the oldest command and shifts all others
4. Creates a copy of the command string with `strdup`
5. Maintains a fixed-size history buffer with a maximum of `MAX_HISTORY` entries

Summary

This shell implementation provides a functional command-line interface with many features found in standard Unix shells:

1. **Input Processing:** Reads and parses user commands.
2. **Command Execution:** Executes both built-in and external commands.
3. **Redirection:** Supports input (`<`), output (`>`), and append (`>>`) redirection.
4. **Pipelines:** Allows connecting commands with pipes (`|`).
5. **Control Operators:** Implements sequential (`;`), logical AND (`&&`), and logical OR (`||`) operators.
6. **Built-in Commands:** Provides basic built-in commands like `cd`, `exit`, `history`.
7. **Signal Handling:** Captures `Ctrl+C` to prevent accidental shell termination.
8. **Command History:** Maintains a history of previous commands.

The implementation uses key system calls like `fork()`, `execvp()`, `pipe()`, and `dup2()` to create processes and set up communication between them, demonstrating core Unix process management concepts.